

PROJECT THEORY REPORT

Quantum Computing Simulator

A From-Scratch Simulation Framework for Learning Qubit Dynamics, Quantum Gates, and Hamiltonian Time Evolution

Course Code: BAPHY106

TEAM

Mathematical Modelling

V N Arunkailash • Siddharth H

Simulator Engine

Yashashwini J • V Keshavan

Graphical User Interface

Niteesh J

Backend & AI Assistant

Miruthusha S J • B Dakshesh

Built entirely from first principles in Python — no Qiskit, Cirq, or NumPy dependency for the core physics engine.

1. Introduction

Quantum computing is often introduced through polished, hardware-abstracted software development kits such as Qiskit, Cirq, and QuTiP. These tools are extremely capable, but their internal machinery — how a qubit's state is actually represented, how a gate operation evolves that state, and how the Schrödinger equation gives rise to the unitary matrices used in a circuit — is hidden behind several layers of abstraction. For a student trying to build genuine intuition, this abstraction is often the biggest obstacle.

This project is a quantum computing simulator built from first principles in Python. Rather than importing an existing quantum framework, every foundational component — complex-number arithmetic, matrix algebra, tensor products, qubit state representation, gate construction, Hamiltonian-driven time evolution, measurement, and even a simple decoherence model — has been implemented from scratch. The objective is not to compete with production-grade simulators on scale or performance, but to make every step of a quantum computation transparent, inspectable, and directly traceable back to the underlying physics.

This report documents the theoretical foundations that motivate and justify the design of the simulator: the mathematics of qubits, the postulates of quantum mechanics that govern their evolution, the distinction between a quantum gate as an idealised matrix versus a quantum gate as a physical, time-dependent process, and the numerical scheme used to approximate that process. It also summarises the resulting system architecture and how the theory maps onto the implemented modules.

2. Motivation and Problem Statement

Existing quantum computing tools are powerful, but they are optimised for production use and research-scale circuit construction rather than conceptual clarity. Several practical gaps motivate this project:

- Beginner-friendly visualization of qubit state evolution is limited in most mainstream frameworks.
- Large, highly-abstracted codebases make it difficult for a learner to inspect or modify the underlying computation.
- Heavy software and hardware requirements restrict experimentation on lightweight or educational setups.
- Simple, human-readable workflow logging for tracking and revisiting experiments is rarely built in.
- Time-dependent Hamiltonian evolution — the actual physical process a gate represents — is usually not emphasised in teaching-oriented simulators, where gates are simply presented as ready-made matrices.

The project addresses these gaps directly by providing a lightweight, transparent, and education-focused simulation environment in which every operation can be traced back to an explicit mathematical or physical justification.

3. Scope of the Project

The simulator covers the representation and evolution of single-qubit and multi-qubit systems, the implementation of the commonly used single- and two-qubit gate set, projective measurement, and the execution of small textbook quantum algorithms — the Deutsch–Jozsa algorithm, the Bernstein–Vazirani

algorithm, and Grover's search algorithm. The emphasis throughout is on conceptual clarity and educational value rather than raw performance or execution on real quantum hardware.

4. Theoretical Background

4.1 The Qubit as a Two-Level Quantum System

A qubit is the quantum analogue of a classical bit. Physically, it is realised as any two-level quantum system — the two lowest energy levels of an atom or ion, the spin-up/spin-down states of an electron, or the two polarisation states of a photon are all common hardware realisations. Mathematically, a qubit's state is a unit vector in a two-dimensional complex Hilbert space, written in Dirac notation as a superposition of the two computational basis states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \text{ with } |\alpha|^2 + |\beta|^2 = 1$$

Here α and β are complex probability amplitudes and the normalisation condition ensures that the total probability of measuring the qubit in one of its two basis states is exactly one. Because α and β are complex numbers rather than real ones, the simulator's very first building block had to be a complete, self-contained implementation of complex-number arithmetic — addition, subtraction, multiplication, division, powers, conjugation, modulus, and phase — since the project deliberately avoids relying on Python's built-in complex type or NumPy for its core physics engine.

4.2 Matrices, State Vectors, and Multi-Qubit Systems

Single-qubit states are represented as 2×1 column vectors, and single-qubit operators (gates) as 2×2 matrices, built on top of a custom matrix class supporting addition, subtraction, multiplication, and other linear-algebra operations over complex entries.

A system of n qubits does not simply consist of n independent 2×2 objects; the qubits live in a joint Hilbert space of dimension 2^n , formed by the tensor (Kronecker) product of the individual single-qubit spaces:

$$|\psi\rangle_{\text{system}} = |\psi\rangle_1 \otimes |\psi\rangle_2 \otimes \cdots \otimes |\psi\rangle_n$$

Gates that act on only a subset of qubits within a larger system are correspondingly expanded to the full 2^n -dimensional space by tensoring the gate with the identity matrix on every qubit it does not act on. This tensor-product construction is what allows the simulator to move from single-qubit dynamics to genuinely multi-qubit, entangled systems and to the two-qubit gates (CNOT, CZ, SWAP) required for the target algorithms.

4.3 Quantum Gates — Two Perspectives

A central theoretical distinction that this project is built around is the difference between how a quantum gate is understood in software and how it is realised in hardware.

Software view: the gate as an ideal matrix

Frameworks such as Qiskit, Cirq, and QuTiP represent every quantum gate as a fixed, instantaneous unitary matrix. Each matrix captures the net effect of some underlying physical process, but the process itself is abstracted away. This is extremely convenient for algorithm design: it lets an algorithm be described purely in terms of logical state transformations, independent of the hardware that will eventually execute it.

Hardware view: the gate as a physical process

In real quantum hardware, a gate is not an instantaneous matrix multiplication. It is implemented by a carefully shaped electromagnetic or laser pulse applied for a finite duration. During that time, the qubit evolves continuously under a physical, generally time-dependent, Hamiltonian. The pulse shape determines how the qubit's energy levels are shifted, how its state vector rotates, and how it interacts with neighbouring qubits. The matrix used in software is therefore the end result of integrating this physical evolution over the pulse duration for one particular Hamiltonian — it is a summary of the physics, not the physics itself.

Because algorithms depend only on the logical sequence of state transformations and not on the specific hardware mechanism producing them, the matrix abstraction is safe to use for algorithm design: it enables hardware-independent reasoning, faster simulation, and portability of the same algorithm across very different physical qubit implementations. This project's simulator supports both views — it uses fixed gate matrices for standard algorithm execution, while also modelling explicit, time-dependent, Hamiltonian-driven evolution so that the origin of those matrices is never hidden from the user.

4.4 Standard Gate Set Implemented

The simulator implements the commonly used single-qubit gates — Identity (I), Pauli-X, Pauli-Y, Pauli-Z, Hadamard (H), Phase (S), and $\pi/8$ (T) — together with the two-qubit gates required for entangling operations and algorithm construction: the Controlled-NOT (in both control/target orientations), Controlled-Z, and SWAP. Every gate is built as an explicit matrix over the custom complex-number and matrix classes described above, keeping the exact numerical construction of each gate fully visible rather than imported from a library.

4.5 The Hamiltonian and the Schrödinger Equation

The Hamiltonian, H , is the operator representing the total energy of a quantum system. It is the single most important object in the theory because it is the generator of time evolution: the time-dependent Schrödinger equation states that the rate of change of a quantum state is governed entirely by the action of the Hamiltonian on that state:

$$i\hbar d/dt |\psi(t)\rangle = H |\psi(t)\rangle$$

In software, the Hamiltonian is simply a matrix used to compute how a state evolves. In hardware, the same Hamiltonian arises physically from applied electromagnetic fields, from interactions between neighbouring qubits, and from intrinsic device properties. This is the same underlying concept expressed in two different representations, and it is precisely the link that mainstream simulators tend to abstract away and that this project puts back at the centre of the design.

4.6 Closed-Form Solution for a Time-Independent Hamiltonian

When the Hamiltonian does not depend on time, the Schrödinger equation has a simple, exact, closed-form solution:

$$|\psi(t)\rangle = e^{-iHt/\hbar} |\psi(0)\rangle$$

where the exponential of the operator H is defined, exactly as for a scalar, through its Taylor series expansion — but understood as a matrix exponential rather than a number. The operator $e^{-iHt/\hbar}$ is called the time evolution operator, often written $U(t)$.

The simplest non-trivial example of this is a single qubit placed in a static field along the z -axis, described by the Hamiltonian $H = (\hbar\omega/2)\sigma_z$, where σ_z is the Pauli-Z matrix and ω is the transition (angular) frequency between the two levels, related to the qubit's energy gap by $\omega = (E_1 - E_2)/\hbar$. Because H is already diagonal in the computational basis, the time evolution operator is:

$$U(t) = \begin{bmatrix} e^{-i\omega t/2} & 0 \\ 0 & e^{+i\omega t/2} \end{bmatrix}$$

Applying $U(t)$ to a general initial state gives the time-evolved amplitudes directly:

$$\alpha(t) = \alpha(0) e^{-i\omega t/2}, \quad \beta(t) = \beta(0) e^{+i\omega t/2}$$

A useful and independently verifiable consequence of this result is that the ratio $\beta(t)/\alpha(t)$ simply accumulates a phase linearly in time:

$$\beta(t)/\alpha(t) = [\beta(0)/\alpha(0)] \cdot e^{i\omega t}$$

Taking the argument (phase angle) of both sides gives a relation that can be inverted to recover the oscillation frequency ω directly from the amplitudes at two different times — a relation used in the project as an analytic consistency check on the numerical time-evolution engine:

$$\begin{aligned} \text{Arg}[\beta(t)/\alpha(t)] &= \text{Arg}[\beta(0)/\alpha(0)] + \omega t \\ \Rightarrow \omega &= (1/t) \{ \text{Arg}[\beta(t)/\alpha(t)] - \text{Arg}[\beta(0)/\alpha(0)] \} \end{aligned}$$

This closed-form check is important for a project like this one: it confirms that the time evolution operator is not an additional physical assumption layered on top of the model, but is simply the exact, provable solution of the

Schrödinger equation for a time-independent Hamiltonian. The simulator's numerical evolution routine is validated against this analytic result for the static-field case before being trusted for the more complex, time-dependent case.

4.7 Time-Dependent Hamiltonians and External Control Fields

Real control of a qubit — the actual mechanism by which a gate is applied — requires a Hamiltonian that depends explicitly on time, since it is built from externally applied, time-varying control pulses. The simulator models a single qubit under general external fields using a Hamiltonian of the form:

$$H(t) = -(\hbar/2) [\omega_0 \sigma_x + \Omega_x(t) \sigma_x + \Omega_y(t) \sigma_y] + E(t) I$$

with the following physical interpretation for each term:

- $\omega_0 = \gamma B_0$ — the Zeeman splitting produced by a static magnetic field B_0 , where γ is the gyromagnetic ratio of the qubit.
- $\Omega_x(t), \Omega_y(t)$ — time-dependent control (drive) fields, the actual pulses used to steer the qubit's state — these are the physical realisation of a software gate.
- $E(t)$ — an electric-field coupling term producing an energy (Stark) shift.
- $\sigma_x, \sigma_y, \sigma_z$ — the Pauli operators, representing the fundamental generators of qubit rotations.

Because $H(t)$ at one instant of time does not, in general, commute with $H(t)$ at another instant when arbitrary control pulses are present, the simple exponential solution of Section 4.6 no longer applies directly, and a more general solution scheme is required — the time-ordered exponential, discussed next.

4.8 Time Ordering, the Dyson Series, and the General Solution

For a time-dependent Hamiltonian, the correct generalisation of the time evolution operator is obtained by dividing the interval into a series of infinitesimal steps and composing the incremental evolution at each step, in strict chronological order. Formally, this leads to the time-ordered exponential, defined through the time-ordering operator T , which takes a product of operators evaluated at different times and rearranges the product so that later-time operators always stand to the left of earlier-time operators:

$$T[\hat{A}(t_1) \hat{B}(t_2)] = \theta(t_1 - t_2) \hat{A}(t_1) \hat{B}(t_2) + \theta(t_2 - t_1) \hat{B}(t_2) \hat{A}(t_1)$$

where θ is the Heaviside step function. Expanding the solution of the Schrödinger equation order by order in this fashion produces the Dyson series, and its resummed, compact form is the defining equation of the time evolution operator for a general, time-dependent Hamiltonian:

$$U(t, t_0) = T \exp\{ -(i/\hbar) \int_{t_0}^t H(t') dt' \}$$

An equivalent and more computationally intuitive way to see the same result is to note that time-ordering arises naturally from applying many small, sequential time translations, one step at a time, and taking the number of steps to infinity while each step shrinks to zero width:

$$| \psi(t) \rangle = \lim_{N \rightarrow \infty} \prod_{j=1}^N [1 - (i/\hbar) H(t' + (j/N)(t-t')) (t-t')/N] | \psi(t') \rangle$$

If the Hamiltonians at different times all commute with one another (as in the static-field case of Section 4.6), the time-ordering symbol becomes redundant and the Dyson series collapses back to the ordinary matrix exponential.

4.9 Discretised Simulation Scheme

The time-ordered exponential cannot, in general, be evaluated in closed form for an arbitrary time-dependent Hamiltonian, so the simulator approximates it numerically. The evolution interval is divided into small steps Δt , over which $H(t)$ is treated as approximately constant, and the evolution operator for the whole interval is approximated as an ordered product of short-time exponentials:

$$U \approx \prod_j \exp[-i H(t_j) \Delta t / \hbar]$$

Each short-time exponential is itself evaluated using a truncated Taylor series expansion of the matrix exponential (rather than full diagonalisation), which keeps the computation simple, transparent, and easy to extend to larger multi-qubit systems. This product-of-exponentials scheme is exactly the discretised, computable form of the time-ordered exponential derived in Section 4.8, and it is the numerical method implemented in the simulator's evolution engine to advance the state of the system step by step across a chosen total simulation time.

4.10 Measurement and Time-Dependent Probability

Quantum measurement in the computational basis is governed by the Born rule: the probability of obtaining outcome $|0\rangle$ or $|1\rangle$ is the squared modulus of the corresponding amplitude at the time of measurement:

$$P_0(t) = |\langle 0 | \psi(t) \rangle|^2 = |\alpha(t)|^2, \quad P_1(t) = |\langle 1 | \psi(t) \rangle|^2 = |\beta(t)|^2$$

The simulator implements this directly: it projects the (possibly time-evolved) state onto each basis vector, normalises the resulting probabilities, and samples a measurement outcome using weighted random selection — collapsing the qubit's state exactly as postulated by the measurement axiom of quantum mechanics.

4.11 Modelling Decoherence: An Open-System Extension

An idealised, closed-system simulator evolves a state under a perfectly unitary operator and never loses information to an external environment. Real qubits, however, are open quantum systems: they interact with their surroundings and gradually lose coherence. To capture this more realistic behaviour, the simulator includes a Monte Carlo quantum-trajectory noise model supporting energy relaxation (T_1 decay), phase damping, and pure dephasing (T_2 processes), together with a simple gate-error channel. At each time step, a stochastic 'quantum jump' may be applied to the state with a probability set by the corresponding decay rate, so that repeated runs of the same nominal circuit produce a statistical ensemble of trajectories rather than a single deterministic outcome — a first step toward realistic, hardware-like simulation while keeping the underlying mechanism fully visible.

4.12 Target Algorithms

The theoretical machinery above is exercised through three canonical algorithms chosen for their pedagogical value in demonstrating quantum parallelism and interference:

- Deutsch–Jozsa algorithm — determines with a single query whether a black-box Boolean function is constant or balanced, illustrating quantum parallelism through superposition and phase kick-back.

- Bernstein-Vazirani algorithm — recovers a hidden bit-string encoded in an oracle in a single query, contrasting sharply with the linear number of queries required classically.
- Grover's search algorithm — provides a quadratic speed-up for unstructured search, illustrating amplitude amplification through repeated oracle and diffusion operations.

5. System Architecture

The simulator follows a modular architecture divided into four functional layers, each with a clear theoretical responsibility. This separation keeps the mapping from physics to code explicit and allows each layer to be extended independently.

Module	Theoretical Responsibility	Owner(s)
Mathematical Operations Engine	Complex-number arithmetic, matrix algebra, tensor products, qubit and system state representation, gate construction, Hamiltonian-driven evolution, measurement	Yashashwini J, Siddharth H, V N Arunkailash
Graphical User Interface	Interactive, beginner-friendly visualisation of qubit states, gate application, and simulation results (built with Python Tkinter)	Niteesh J
Logging System	Custom, file-based, dependency-free recording of every operation performed, with automatic log rotation, for reproducibility and debugging	Miruthusha S J
AI Assistant Module	An integrated AI-chat panel (built on a conversational API) that lets users ask conceptual and usage doubts about the simulator and get context-aware answers in real time	B Dakshesh

5.1 Mapping Theory onto Implementation

The theoretical layers described in Section 4 correspond directly to concrete components in the codebase:

- A from-scratch complex-number class (supporting addition, multiplication, division, conjugation, modulus, and phase) provides the amplitudes α and β that appear throughout Section 4.
- A from-scratch matrix class provides the linear algebra — including the tensor product — needed to represent state vectors, gates, and multi-qubit systems (Sections 4.1–4.2).
- A gate library builds the standard single- and two-qubit gate matrices (I, X, Y, Z, H, S, T, CNOT, CZ, SWAP) directly on top of the matrix and complex-number classes (Section 4.4).
- A single-qubit class holds a qubit's energy levels, derives its transition frequency ω , normalises its initial amplitudes, and stores symbolic (algebraic) representations of its local and driving Hamiltonians — the object-level realisation of Sections 4.6–4.7.
- A system class composes multiple qubits via the tensor product, assembles the total time-dependent Hamiltonian for the whole system, and evolves it step-by-step using the truncated Taylor-series product scheme of Section 4.9, optionally overlaying the Monte Carlo decoherence model of Section 4.11.
- A logging class instruments key methods with a call-tracing decorator, writing a rotating, dependency-free log file so that every operation performed during a session can be reviewed or replayed later.
- A Tkinter-based GUI layer exposes qubit states, gate operations, and simulation controls to the user, with an optional in-app chat panel for guided, context-aware assistance.

6. Tools and Technologies

Python is the primary language, chosen for its readability and the speed with which the physics engine could be developed and iterated on. The physics engine itself rests on linear algebra, complex-number arithmetic, and the numerical integration of a first-order differential equation (the Schrödinger equation) — no external quantum SDK, and no NumPy, is used in the core computation. The simulator is additionally paired with an AI-based chat assistant, built on a conversational API, so that users can ask conceptual or usage doubts and receive context-aware answers directly within the simulation environment.

7. Comparison with Existing Frameworks (Qiskit)

Qiskit is an excellent, industry-standard SDK: it offers mature circuit-construction tools, direct access to real IBM Quantum hardware, and a highly optimised, scalable execution backend built for production and research use. However, in achieving this, Qiskit — like most mainstream frameworks — deliberately abstracts away the underlying physics: Hamiltonians, continuous time evolution, and the physical origin of each gate are not directly visible to the user, and gates are treated as ideal, instantaneous, hardware-independent operations. This makes Qiskit outstanding for algorithm development, but less suited to building physical intuition.

Aspect	Qiskit / Mainstream SDKs	This Project
Gate representation	Fixed, instantaneous unitary matrix	Matrix form and explicit Hamiltonian-driven derivation, both available
Time evolution	Abstracted away	Modelled explicitly via the Schrödinger equation and time-ordered exponential
Dependencies	Large SDK, many dependencies	Minimal — core physics engine has none
Target audience	Researchers, production users	Learners building conceptual intuition
Hardware access	Real IBM Quantum hardware	Not applicable (educational simulation only)
Transparency	Internal mechanics hidden	Every operation traceable to first principles

This project does not aim to replace Qiskit; it aims to complement it by making the physics Qiskit abstracts away visible and directly modifiable, so that a learner can move on to production frameworks with a clearer mental model of what those frameworks are actually computing underneath their circuit diagrams.

8. Advantages of the Simulator

Lightweight design

- Minimal dependencies keep setup and execution fast.
- Suitable for small-system simulations on modest, low-resource hardware.

Simpler learning interface

- A beginner-friendly GUI makes qubit states and gate operations easy to visualise.
- The transparent implementation helps build genuine physical intuition rather than pattern-matching on circuit diagrams.

Open-source advantage

- Fully customisable and extensible by anyone studying the code.
- Encourages collaborative learning, experimentation, and further development.

9. Expected Outcomes

By the end of the project, a functional, open-source quantum computing simulator is expected to allow users to observe qubit states before and after the application of quantum gates, enabling step-by-step analysis of quantum state evolution. Releasing the project as open source turns the simulator into a collaborative learning platform that others can study, modify, and extend — improving conceptual understanding of quantum computing while providing a scalable foundation for future enhancements such as additional gate sets, larger multi-qubit systems, and richer noise modelling.

10. Challenges and Limitations

- Implementing complex-number arithmetic, linear algebra, and Hamiltonian evolution entirely from scratch is significantly more effort-intensive than relying on established libraries such as NumPy or Qiskit.
- Simulation cost grows exponentially with the number of qubits, since the state space has dimension 2^n ; this fundamentally limits the system sizes that can be simulated on ordinary hardware, regardless of implementation efficiency.
- Numerically approximating the time-ordered exponential via a truncated, discretised product of matrix exponentials introduces a trade-off between step size (accuracy) and computation time.
- The project targets conceptual clarity rather than performance, so it is not intended to compete with production simulators on scale or speed.

11. Future Scope

- Support for additional, more advanced quantum gates and parametrised gate families.
- Simulation of larger multi-qubit systems through further performance tuning of the pure-Python computational core.
- Richer, more physically grounded noise and decoherence modelling beyond the current Monte Carlo scheme.
- Expanded visualisation tools, including graphical Bloch-sphere-style representations of qubit states over time.
- Broader algorithm coverage beyond Deutsch–Jozsa, Bernstein–Vazirani, and Grover's search.

12. Conclusion

This project set out to answer a simple question: what does a quantum gate actually do, underneath the matrix? By building every layer — complex numbers, matrices, tensor products, gates, Hamiltonians, time evolution, measurement, and even a basic noise model — from first principles, the simulator keeps that question answerable at every step. The result is a lightweight, transparent, and extensible educational tool that sits alongside frameworks like Qiskit rather than competing with them: where Qiskit optimises for scale and production use, this project optimises for the conceptual clarity that comes from never letting the physics disappear behind the abstraction.

13. References

1. Griffiths, D. J., Introduction to Quantum Mechanics — standard treatment of the time-dependent Schrödinger equation and the time evolution operator.
2. Nielsen, M. A. and Chuang, I. L., Quantum Computation and Quantum Information — standard reference for qubits, quantum gates, and quantum algorithms.
3. Sakurai, J. J., Modern Quantum Mechanics — treatment of the Dyson series and the time-ordered exponential.
4. IBM Qiskit Documentation — reference framework used for architectural and conceptual comparison.